

오늘의 작업기록 — 2026년 7월 1일

mini-ledger — 금액 필터 조회(getHistoryAbove) + IntelliJ 복귀·2-머신 동기화

(30분짜리 세션 — 짧아도 필터링 공식을 한 번 더 굴리고, 두 기계를 다시 맞춘 날)

한눈에

항목	내용
목표	(30분 제약) 환불 대신 가벼운 금액 필터 조회로 공식 굳히기 + 잔디
결과	완료 ✓ — 1000원 이상만 출력(200·500·300 빠짐) 확인
커밋	add getHistoryAbove for amount filtering (44f09cf)
저장소	github.com/Taewoo-Won/mini-ledger
오늘의 핵심	필터링 공식 2회차 + IntelliJ 복귀·동기화 파이프라인 정립
새 개념	문자열= .equals() vs 숫자=부등호(>=) 자리·형태 차이 · git stash/pull/alias
환경	컴(IntelliJ) 코딩 복귀 · 잔디는 폰/VPS (원칙 유지)
학습 방식	어제 공식을 조건만 바꿔 재사용. "왜 minAmount 자리가 다르지?" 의문을 끝까지 청산

일정이 있어 30분밖에 없던 날. 환불은 설계가 무거워(원본 지목·이중환불·반대거래) 반쪽으로 꿩길 위험이 커서 미뤘고, 대신 어제 필터링 공식을 한 번 더 굴리는 금액 필터로 잡았다. 짧아도 (1) 공식 2회차로 손에 굳히고, (2) 밀렸던 두 기계(본진↔폰) 동기화를 깔끔히 정립했다.

0. 출발점 / 오늘의 목표

- 이전까지: 증분 1~5 + 이력 조회(getHistoryByAccount)까지. mini-ledger 7개 .java.
- 오늘 목표(30분): 금액 필터 조회 — 전체 history에서 특정 금액 이상만. 필터링 공식 재사용 + 잔디.
- 왜 이것: 환불(다음 본작업)은 1~1.5시간 필요한 설계 난제. 30분엔 부적합. 대신 어제 배운 필터링을 굳히는 위망업으로.
- 다음: 환불(refund) — 두 번째 솔로 런으로 제대로. 시간 여유 있는 세션에.
- 작업 도구: IntelliJ(사지방 본진, 코딩) + 폰/VPS(잔디). 컴 복귀 첫날.

PART 1 — 환경 복귀: 2-머신 동기화 정립 (오늘의 숨은 수학)

컴(IntelliJ) 복귀 첫날이라, 본진(mini-ledger-main)이 폰/VPS로 올린 최신 커밋들보다 뒤쳐져 있었다. 이걸 맞추는 과정을 정립했다.

1-1. 문제 진단

- 본진 clone은 "그 시점 스냅샷"이라, 이후 폰/VPS로 올린 5개 커밋(데이터모델·감사로그·역등성·조회·README)이 로컬에 없었다.
- git log --oneline 로 확인: 본진 HEAD가 532825f (커스텀 예외)에 멈춤. 원격은 ee9f03b (조회).

1-2. 충돌과 해결

- git pull 시도 → 예라: 로컬에 커밋 안 된 Transaction.java/TransactionType.java(예전에 만들다 만 것)가 있어 덮어쓰기 거부. → 이걸 git의 안전장치. 조용히 덮지 않고 멈춰 물어본 것.
- git stash — 커밋 안 된 로컬 변경을 안전하게 옆으로 치움(삭제 아님). → git pull 성공(Fast-forward, 5파일 갱신).

1-3. 앞으로 — git sync 한 단어

매번 stash→pull→확인 수동은 과함. alias로 묶음:

```
git config --global alias.sync '!git stash && git pull && git log --oneline -5'
```

→ 이후 동기화는 git sync 한 단어. (충돌 나면 중간에 멈춰 알려주니 안전장치는 유지.)

★ 판단: 완전 자동 동기화는 하지 않는다. 오늘 그 충돌이 자동이었으면 로컬 작업을 조용히 덮었을 수 있다. git이 멈춰 물어본 게 안전이다. "작업 전 git sync 한 번"은 프로 표준 습관(실무에서도 작업 전 pull부터). 자동화로 없앨 게 아니라 한 줄로 줄이는 게 맞다.

PART 2 — 금액 필터 조회 (필터링 공식 2회차)

2-1. 5단계 (어제 공식 재사용이라 빠르게)

단계	내용	어제 대비
① 데이터	없음 (history 재사용)	동일
② 입구	Bank에 <code>getHistoryAbove(long minAmount)</code>	동일 원칙(데이터가 사는 Bank)
③ 로직	빈 바구니 → 순회 → if(금액) → 담기 → 반환	조건만 다름
④ 호출	Main에서 출력	동일
⑤ 확인	1000 이상만 나오면 성공	동일

→ 어제 언어화한 **필터링 공식(빈 바구니→순회→if→담기→반환)**이 조건만 바뀌 그대로 작동. "조건만 바꾸면 뭐든 거른다"를 손으로 증명.

2-2. 코드

```
public List<Transaction> getHistoryAbove(long minAmount) {
    List<Transaction> result = new ArrayList<>();
    for (Transaction t : history) {
        if (t.getAmount() >= minAmount) {
            result.add(t);
        }
    }
    return result;
}
```

Main 호출:

```
System.out.println("===== 1000원 이상 거래 =====");
for (Transaction t : bank.getHistoryAbove(1000)) {
    System.out.println(t.getType() + " " + t.getFrom() + " -> " + t.getTo() + " : " + t.getAmount());
}
```

2-3. 실행 결과 (⑤ 검증)

```
===== 1000원 이상 거래 =====
TRANSFER a -> b : 1000
```

→ 1000짜리 하나만. 200(이체)·500(입금)·300(출금) 전부 걸러짐. 필터 작동. (999999는 실패라 애초에 기록 없음.)

PART 3 — "왜?" 문답 (오늘 청산한 것)

Q. 어제는 `if (id.equals(...))` 로 시작했는데, 왜 오늘은 `minAmount` 가 아니라 `t.getAmount()` 가 앞에 오나?

핵심 오해: "자리가 다르다"가 아니라 **"비교 방식이 다르다"**였다.

- ① **자리는 사실 같다.** 어제도 오늘도 "기준 vs 각 거래 값"을 비교한다. - 어제: `id.equals(t.getFrom())` → 기준=id, 각 거래 값=t.getFrom() - 오늘: `t.getAmount() >= minAmount` → 각 거래 값=t.getAmount(), 기준=minAmount - minAmount 안 빼졌다 — 오른쪽에 있다. **순서는 상관없다** (`A >= B` = `B <= A`).
- ② **진짜 차이 = 문자열이나 숫자냐.** - 문자열(id/from/to)은 `.equals()` 메서드로 비교 → `A.equals(B)` 형태(점+괄호). - 숫자(amount/minAmount)는 **부등호 기호**로 비교 → `A >= B` 형태(사이 기호). 점도 괄호도 없음. - → 어제 규칙("문자열>equals, 숫자=부등호/==")이 **문법 모양 자체를 바꾼다**.
- ③ **왜 t.getAmount()가 필요한가.** - minAmount = 고정 기준선(1000). / t.getAmount() = 각 거래의 실제 금액(반복마다 200,1000,500...). - 거르려면 **각 거래 값(변함)을 기준(고정)에 맞대야** 한다. 기준만 두 번 쓰면(`minAmount >= minAmount`) 항상 참이라 필터가 안 됨.

정리: 어제 `id.equals(t.getXXX)` → 오늘 `t.getAmount() >= minAmount` . 둘 다 **"기준 vs 각 거래 값"** 구조. 문자열이라 equals였던 게, 숫자라 부등호로 바뀐 것뿐.

PART 4 — 오늘 지킨 원칙 (환경/판단)

- 코딩은 컴, 잔디는 폰/VPS. 컴에서 push가 "빠르긴 한데" 유혹이 있었으나 — **push는 인증을 공용 PC에 남긴다**(pull/fetch는 읽기라 인증 불필요했지만 push는 쓰기라 필요). 몇 분 빼름 < 공용 PC 보안. 원칙 유지. 코드는 EOF로 폰/VPS에 옮겨 거기서 커밋·push.
- **대시보드 재설계는 하지 않는다.** (세 번 막힌 자리. 오버엔지니어링 + 추적 도구 외주라 스크프 밖. 컴 복귀했다고 정당해지지 않음 — 판단 기준은 환경이 아니라 학습 원칙.)
- 폰 EOF 복붙은 여전히 깨진다 → 항상 `cat` 으로 검증 후 진행. (오늘도 첫 붙여넣기 깨졌고 재시도로 통과. 마지막 cat이 진실.)

5. 산출물

- 파일: `Bank.java` (getHistoryAbove 추가), `Main.java` (호출 추가). 나머지 변경 없음.
- 검증: 1000 이상만 1건(TRANSFER 1000). 실행 exit code 0.
- 커밋: `44f09cf` "add getHistoryAbove for amount filtering" (2 files, +14). `ee9f03b` . `44f09cf` .
- 커밋 계보: `fb3897b` (감사로그) → `d82e58e` (명등성) → `ee9f03b` (계좌조회) → **`44f09cf`** (금액조회)

6. 개념 사전

- **필터링 공식(재확인):** 빈 바꾸니 → 전체 순회 → if(조건) → add → return. **조건만 같아끼우면 어떤 필터든** (계좌·금액·타입·기간...).
- **문자열 비교 vs 숫자 비교:** 문자열= `.equals()` (점+괄호). 숫자=부등호(`>=` , `<` , `==` , 사이 기호). 형태가 다르다.
- **git stash:** 커밋 안 된 로컬 변경을 임시 보관(삭제 아님). pull 충돌 회피용.
- **git alias:** 긴 명령을 한 단어로. `git sync` = stash+pull+log. (로컬 설정, 인증정보 아님.)
- **pull 충돌 안전장치:** 로컬 미커밋 변경을 덮어쓸 상황이면 git이 멈춰 물어봄. 자동 아닌 게 안전.

7. 비유 모음

- **필터링 공식 = 만능 체:** 체(공식)는 그대로, 구멍 크기(조건)만 바꾸면 콩·모래·자갈 뭐든 거른다. 어제는 "계좌" 구멍, 오늘은 "금액" 구멍.
- **git stash = 책상 서랍:** 하던 일(미커밋 변경)을 잠깐 서랍에 넣고 책상 치우고(pull), 필요하면 다시 꺼낸다.
- **equals vs 부등호 = 이름표 vs 저울:** 문자열은 이름표 맞대보기(equals), 숫자는 저울에 올려 크기 재기(>=).

8. 검증 질문 (다음날 복기용 — 답 적지 말 것)

1. 필터링 공식 5조각을 순서대로. `getHistoryByAccount` 와 `getHistoryAbove` 의 **유일한** 차이는?
2. 문자열은 `.equals()` , 숫자는 `>=` 를 쓰는 이유? 형태(점+괄호 vs 사이 기호)가 왜 다른가?
3. `t.getAmount() >= minAmount` 에서 minAmount만 두 번 쓰면 왜 안 되나?
4. `git pull` 이 에러를 내며 멈춘 이유는? 그게 왜 "안전장치"인가? `git stash` 는 무엇을 하나?
5. 코딩은 컴에서 하면서 push는 폰/VPS로 하는 이유는?

9. 회고

30분짜리 짧은 세션. 근데 두 가지를 챙겼다. ① **필터링 공식 2회차** — 어제 언어화한 공식을 조건만 바꿔 재사용하며 "만능 체"임을 손으로 확인했다. 특히 "왜 minAmount 자리가 어제 id랑 다르지?"라는 의문을 그냥 넘기지 않고 — 자리가 아니라 **문자열/숫자 비교법의 차이**임을 끝까지 쫓았다. 이런 작은 의문을 청산하는 습관이 복리로 쌓인다.

② **2-머신 동기화 정립.** 컴 복귀 첫날 밀린 커밋 충돌을 stash→pull로 풀고, `git sync` alias로 앞으로를 한 단어화했다. 그리고 "완전 자동은 안 한다"는 판단 — 오늘 충돌이 자동이었으면 로컬을 조용히 덮었을 것. git이 멈춰 물어본 게 안전이라는 걸 몸으로 봤다.

원칙도 지켰다. 컴 push가 빨랐지만 "잔디는 폰/VPS"를 안 썼고, 대시보드 재설계 유혹도 스코프 밖으로 막았다. 짧은 날일수록 원칙이 시험받는데, 오늘은 통과다. 환불은 다음에 온전히 — 두 번째 솔로 런으로 제대로 씹는다.

한 줄: 30분이었지만 만능 체(필터링 공식)를 한 번 더 굴리고, 두 기계를 `git sync` 한 단어로 잇고, 원칙 세 개를 지킨 날.